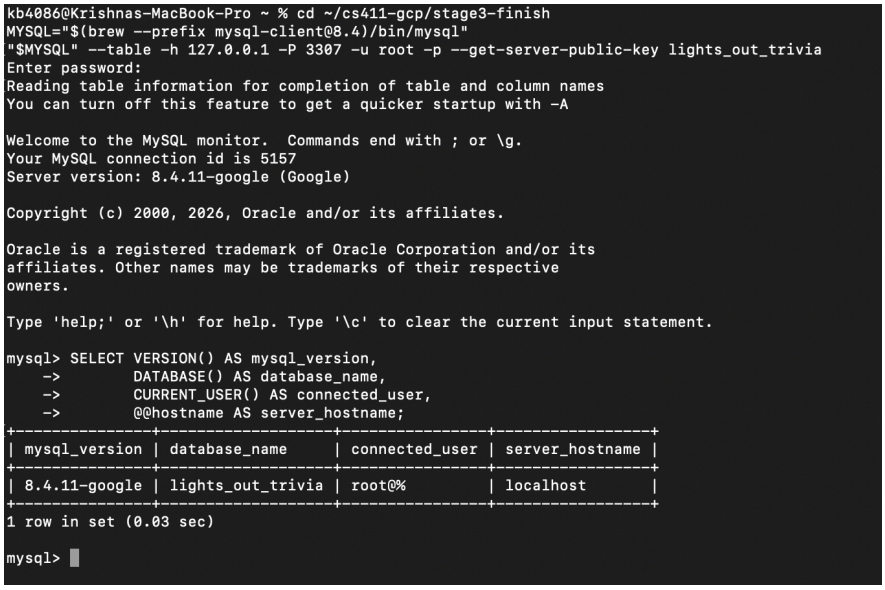
CS 411 PT1 Stage 3: Database Implementation and Indexing

Team 058 | Lights Out Trivia | October 30, 2026

1. Database implementation

- We implemented Lights Out Trivia in Google Cloud SQL using MySQL 8.4.11-google.
- The database, lights_out_trivia, contains 21 tables.
- Twelve tables contain imported Formula 1 records, totaling 634,176 rows. Five tables have more than 1,000 records: Qualifying (10,494), DriverStanding (1,324), RaceResult (19,235), Lap (589,081), and PitStop (11,371). The remaining tables support application functionality, including questions, quizzes, answer options, attempts, and saved questions.



```
kb4086@Krishnas-MacBook-Pro ~ % cd ~/cs411-gcp/stage3-finish
MYSQL="$(brew --prefix mysql-client@8.4)/bin/mysql"
"$MYSQL" --table -h 127.0.0.1 -P 3307 -u root -p --get-server-public-key lights_out_trivia
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 5157
Server version: 8.4.11-google (Google)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SELECT VERSION() AS mysql_version,
    ->        DATABASE() AS database_name,
    ->        CURRENT_USER() AS connected_user,
    ->        @@hostname AS server_hostname;
+---------------+------------------+----------------+-----------------+
| mysql_version | database_name    | connected_user | server_hostname |
+---------------+------------------+----------------+-----------------+
| 8.4.11-google | lights_out_trivia | root@%         | localhost       |
+---------------+------------------+----------------+-----------------+
1 row in set (0.03 sec)

mysql>
```

```
mysql> SELECT 'Season' AS table_name, COUNT(*) AS row_count FROM Season
    -> UNION ALL
    -> SELECT 'Location', COUNT(*) FROM Location
    -> UNION ALL
    -> SELECT 'Circuit', COUNT(*) FROM Circuit
    -> UNION ALL
    -> SELECT 'Driver', COUNT(*) FROM Driver
    -> UNION ALL
    -> SELECT 'Constructor', COUNT(*) FROM Constructor
    -> UNION ALL
    -> SELECT 'Race', COUNT(*) FROM Race
    -> UNION ALL
    -> SELECT 'Qualifying', COUNT(*) FROM Qualifying
    -> UNION ALL
    -> SELECT 'DriverStanding', COUNT(*) FROM DriverStanding
    -> UNION ALL
    -> SELECT 'ConstructorStanding', COUNT(*) FROM ConstructorStanding
    -> UNION ALL
    -> SELECT 'RaceResult', COUNT(*) FROM RaceResult
    -> UNION ALL
    -> SELECT 'Lap', COUNT(*) FROM Lap
    -> UNION ALL
    -> SELECT 'PitStop', COUNT(*) FROM PitStop;
+--------------------+-----------+
| table_name         | row_count |
+--------------------+-----------+
| Season             |        46 |
| Location           |        75 |
| Circuit            |        77 |
| Driver             |       861 |
| Constructor        |       212 |
| Race               |       812 |
| Qualifying         |     10494 |
| DriverStanding     |      1324 |
| ConstructorStanding |       588 |
| RaceResult         |     19235 |
| Lap                |    589081 |
| PitStop            |     11371 |
+--------------------+-----------+
12 rows in set (0.49 sec)
```

The full CREATE TABLE statements are in ***ddl.sql*** and Appendix A.

2. Three selected advanced SQL queries

Q1. Circuit Kings

Circuit Kings finds the drivers with the most race wins at each circuit. It joins RaceResult, Race, Circuit, and Driver, groups results by circuit and driver, and uses a HAVING subquery to retain the leading drivers. We ran the query with LIMIT 15 and verified that it returned 15 rows, starting with Lewis Hamilton's nine wins at Silverstone Circuit.

```
SELECT c.name    AS circuit,
       d.forename,
       d.surname,
       COUNT(*)  AS wins
FROM RaceResult rr
JOIN Race    r ON r.raceId    = rr.raceId
JOIN Circuit c ON c.circuitId = r.circuitId
```

```
JOIN Driver  d ON d.driverId  = rr.driverId
WHERE rr.finishPosition = 1
GROUP BY c.circuitId, c.name, d.driverId, d.forename, d.surname
HAVING COUNT(*) = (SELECT COUNT(*)
                   FROM RaceResult rr2
                   JOIN Race r2 ON r2.raceId = rr2.raceId
                   WHERE rr2.finishPosition = 1
                     AND r2.circuitId = c.circuitId
                   GROUP BY rr2.driverId
                   ORDER BY COUNT(*) DESC
                   LIMIT 1)
ORDER BY wins DESC, circuit
LIMIT 15;
```

evidence_label
Q1 Circuit Kings – TOP 15 RESULTS

circuit	forename	surname	wins
Silverstone Circuit	Lewis	Hamilton	9
Circuit de Nevers Magny-Cours	Michael	Schumacher	8
Hungaroring	Lewis	Hamilton	8
Autodromo Enzo e Dino Ferrari	Michael	Schumacher	7
Circuit Gilles Villeneuve	Lewis	Hamilton	7
Circuit Gilles Villeneuve	Michael	Schumacher	7
Circuit de Barcelona-Catalunya	Michael	Schumacher	6
Circuit de Barcelona-Catalunya	Lewis	Hamilton	6
Circuit de Monaco	Ayrton	Senna	6
Circuit de Spa-Francorchamps	Michael	Schumacher	6
Shanghai International Circuit	Lewis	Hamilton	6
Suzuka Circuit	Michael	Schumacher	6
Autódromo Hermanos Rodríguez	Max	Verstappen	5
Autódromo Internacional Nelson Piquet	Alain	Prost	5
Autodromo Nazionale di Monza	Michael	Schumacher	5

Q3. Pit Stop Performance

Pit Stop Performance compares a constructor's average pit stop duration with the average for the same season. It joins PitStop, RaceResult, Race, and Constructor, groups by season and constructor, and compares the aggregate with a season-average subquery. We exclude stops lasting 60 seconds or more and require at least 20 stops per constructor.

```
SELECT r.year,
       con.name                          AS constructor,
```

```
       COUNT(*)                          AS stops,
                     ROUND(AVG(ps.durationMs) / 1000, 3) AS avgStopSec
FROM PitStop ps
JOIN RaceResult  rr  ON rr.raceId = ps.raceId AND rr.driverId =
ps.driverId
JOIN Race        r   ON r.raceId  = ps.raceId
JOIN Constructor con ON con.constructorId = rr.constructorId
WHERE ps.durationMs < 60000
GROUP BY r.year, con.constructorId, con.name
HAVING COUNT(*) >= 20
   AND AVG(ps.durationMs) < (SELECT AVG(ps2.durationMs)
                             FROM PitStop ps2
                             JOIN Race r2 ON r2.raceId = ps2.raceId
                             WHERE r2.year = r.year
                               AND ps2.durationMs < 60000)
ORDER BY r.year DESC, avgStopSec
LIMIT 15;
```

evidence_label
Q3 Pit Stop Performance — TOP 15 RESULTS

year	constructor	stops	avgStopSec
2024	McLaren	77	23.921
2024	Alpine F1 Team	71	24.221
2024	Mercedes	78	24.250
2024	Ferrari	75	24.301
2024	Red Bull	80	24.357
2024	Aston Martin	81	24.713
2024	RB F1 Team	72	24.734
2023	Ferrari	69	23.663
2023	Mercedes	80	23.882
2023	Red Bull	88	23.928
2023	Aston Martin	80	24.129
2023	AlphaTauri	83	24.539
2022	Red Bull	83	23.952
2022	Alpine F1 Team	68	24.000
2022	Aston Martin	75	24.163

Q4. Fastest Lap Specialists

Fastest Lap Specialists counts the number of fastest laps recorded by each driver. The query calculates the minimum lap time for each race and joins those results back to Lap to identify the drivers. It then groups the results by driver to count their fastest laps.

```
SELECT d.forename, d.surname, COUNT(*) AS fastestLaps,
       MIN(r.year) AS firstYear, MAX(r.year) AS lastYear
FROM Lap l
JOIN (SELECT raceId, MIN(lapTimeMs) AS bestLapTimeMs
      FROM Lap GROUP BY raceId) best
  ON best.raceId = l.raceId AND best.bestLapTimeMs = l.lapTimeMs
JOIN Race r ON r.raceId = l.raceId
JOIN Driver d ON d.driverId = l.driverId
GROUP BY d.driverId, d.forename, d.surname
ORDER BY fastestLaps DESC, d.surname LIMIT 15;
```

```
+-------------------------------------------------+
| evidence_label                                  |
+-------------------------------------------------+
| Q4 Fastest Lap Specialists - REWRITTEN SQL - TOP 15 |
+-------------------------------------------------+
+-----------+---------------+-------------+-----------+----------+
| forename  | surname       | fastestLaps | firstYear | lastYear |
+-----------+---------------+-------------+-----------+----------+
| Lewis     | Hamilton      |          66 |      2007 |     2024 |
| Michael   | Schumacher    |          54 |      1996 |     2012 |
| Kimi      | Räikkönen     |          45 |      2002 |     2018 |
| Sebastian | Vettel        |          38 |      2009 |     2019 |
| Max       | Verstappen    |          35 |      2016 |     2024 |
| Fernando  | Alonso        |          26 |      2003 |     2024 |
| Mika      | Häkkinen      |          25 |      1997 |     2001 |
| Nico      | Rosberg       |          20 |      2006 |     2016 |
| Valtteri  | Bottas        |          19 |      2014 |     2021 |
| Mark      | Webber        |          19 |      2009 |     2013 |
| Rubens    | Barrichello   |          17 |      2000 |     2009 |
| Daniel    | Ricciardo     |          17 |      2014 |     2024 |
| Felipe    | Massa         |          15 |      2006 |     2014 |
| David     | Coulthard     |          14 |      1997 |     2002 |
| Juan      | Pablo Montoya |          13 |      2001 |     2005 |
+-----------+---------------+-------------+-----------+----------+
```

3. Indexing experiments

For each of the three queries, we compared the baseline execution plan using existing indexes with three additional index designs using EXPLAIN ANALYZE. We dropped each experimental index before testing the next design. We compared the estimated cost of the outermost nested-loop join in each plan.

Q1. Circuit Kings

Design	Index	Cost	Time	Change in cost
Baseline	Existing indexes	3,507	106.0 ms	reference
Design A	RaceResult(finishPosition)	936	32.4ms	73.3% reduction
Design B	RaceResult(finishPosition, raceId)	936	32.3ms	73.3% reduction
Design C	RaceResult(finishPosition, driverId, raceId)	937	101.0ms	73.3% reduction

Baseline: With no index on finishPosition, MySQL reaches RaceResult through its primary key. It reads every result for each race and then keeps only the winners (finishPosition = 1).

Design A allows MySQL to jump straight to the winning rows. Every index also stores the primary key columns (raceId, driverId), so this index contains every RaceResult column the query uses. MySQL never has to read the table rows themselves (a covering index lookup). The estimated cost drops from 3,507 to 936 (−73.3%).

Designs B and C give no further gain. B's extra raceId column is already stored in A's index as part of the primary key, so B is effectively the same index, with the same cost. C also adds driverId, and its cost (937) is practically identical.

We selected Design A because it achieved the lowest estimated cost while requiring only one explicitly indexed column, finishPosition. Designs B and C did not provide any additional cost benefit.

```
kb4086@Krishnas-MacBook-Pro stage3-finish % cat doc/evidence/indexing/plans/q1_baseline.txt
Q1 BASELINE
MySQL 8.4 EXPLAIN ANALYZE, captured from original experiment run.

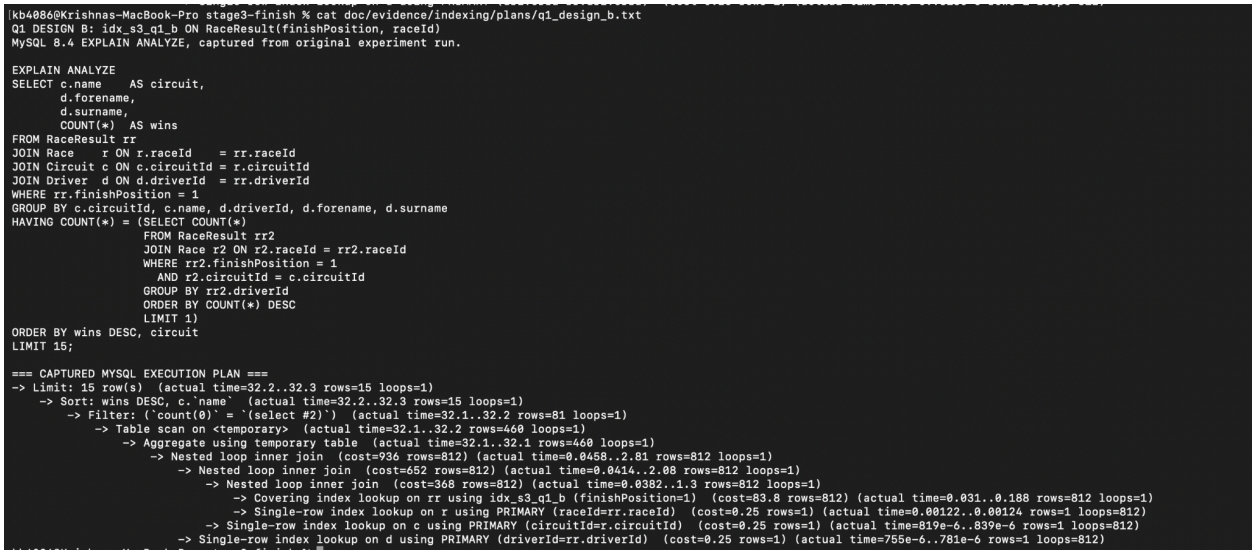
EXPLAIN ANALYZE
SELECT c.name    AS circuit,
       d.forename,
       d.surname,
       COUNT(*)  AS wins
FROM RaceResult rr
JOIN Race    r ON r.raceId    = rr.raceId
JOIN Circuit c ON c.circuitId = r.circuitId
JOIN Driver  d ON d.driverId  = rr.driverId
WHERE rr.finishPosition = 1
GROUP BY c.circuitId, c.name, d.driverId, d.forename, d.surname
HAVING COUNT(*) = (SELECT COUNT(*)
                   FROM RaceResult rr2
                   JOIN Race r2 ON r2.raceId = rr2.raceId
                   WHERE rr2.finishPosition = 1
                     AND r2.circuitId = c.circuitId
                   GROUP BY rr2.driverId
                   ORDER BY COUNT(*) DESC
                   LIMIT 1)
ORDER BY wins DESC, circuit
LIMIT 15;

=== CAPTURED MYSQL EXECUTION PLAN ===
-> Limit: 15 row(s)  (actual time=106..106 rows=15 loops=1)
    -> Sort: wins DESC, c.`name`  (actual time=106..106 rows=15 loops=1)
        -> Filter: (`count(0)` = `(select #2)`)  (actual time=106..106 rows=81 loops=1)
            -> Table scan on <temporary>  (actual time=106..106 rows=460 loops=1)
                -> Aggregate using temporary table  (actual time=106..106 rows=460 loops=1)
                    -> Nested loop inner join  (cost=3507 rows=2164) (actual time=0.0586..8.05 rows=812 loops=1)
                        -> Nested loop inner join  (cost=2749 rows=2164) (actual time=0.0537..7.17 rows=812 loops=1)
                            -> Nested loop inner join  (cost=367 rows=812) (actual time=0.0407..0.425 rows=812 loops=1)
                                -> Covering index scan on r using circuitId  (cost=83.2 rows=812) (actual time=0.0336..0.151 rows=812 loops=1)
                                -> Single-row index lookup on c using PRIMARY (circuitId=r.circuitId)  (cost=0.25 rows=1) (actual time=192e-6..212e-6 rows=1 loops=812)
                            -> Filter: (rr.finishPosition = 1)  (cost=0.268 rows=2.67) (actual time=0.00566..0.00818 rows=1 loops=812)
                                -> Index lookup on rr using PRIMARY (raceId=r.raceId)  (cost=0.268 rows=26.7) (actual time=0.00455..0.00702 rows=23.7 loops=812)
                        -> Single-row index lookup on d using PRIMARY (driverId=rr.driverId)  (cost=0.25 rows=1) (actual time=932e-6..957e-6 rows=1 loops=812)
```

```
kb4086@Krishnas-MacBook-Pro stage3-finish % cat doc/evidence/indexing/plans/q1_design_a.txt
Q1 DESIGN A: idx_s3_q1_a ON RaceResult(finishPosition)
MySQL 8.4 EXPLAIN ANALYZE, captured from original experiment run.

EXPLAIN ANALYZE
SELECT c.name    AS circuit,
       d.forename,
       d.surname,
       COUNT(*)  AS wins
FROM RaceResult rr
JOIN Race    r ON r.raceId    = rr.raceId
JOIN Circuit c ON c.circuitId = r.circuitId
JOIN Driver  d ON d.driverId  = rr.driverId
WHERE rr.finishPosition = 1
GROUP BY c.circuitId, c.name, d.driverId, d.forename, d.surname
HAVING COUNT(*) = (SELECT COUNT(*)
                   FROM RaceResult rr2
                   JOIN Race r2 ON r2.raceId = rr2.raceId
                   WHERE rr2.finishPosition = 1
                     AND r2.circuitId = c.circuitId
                   GROUP BY rr2.driverId
                   ORDER BY COUNT(*) DESC
                   LIMIT 1)
ORDER BY wins DESC, circuit
LIMIT 15;

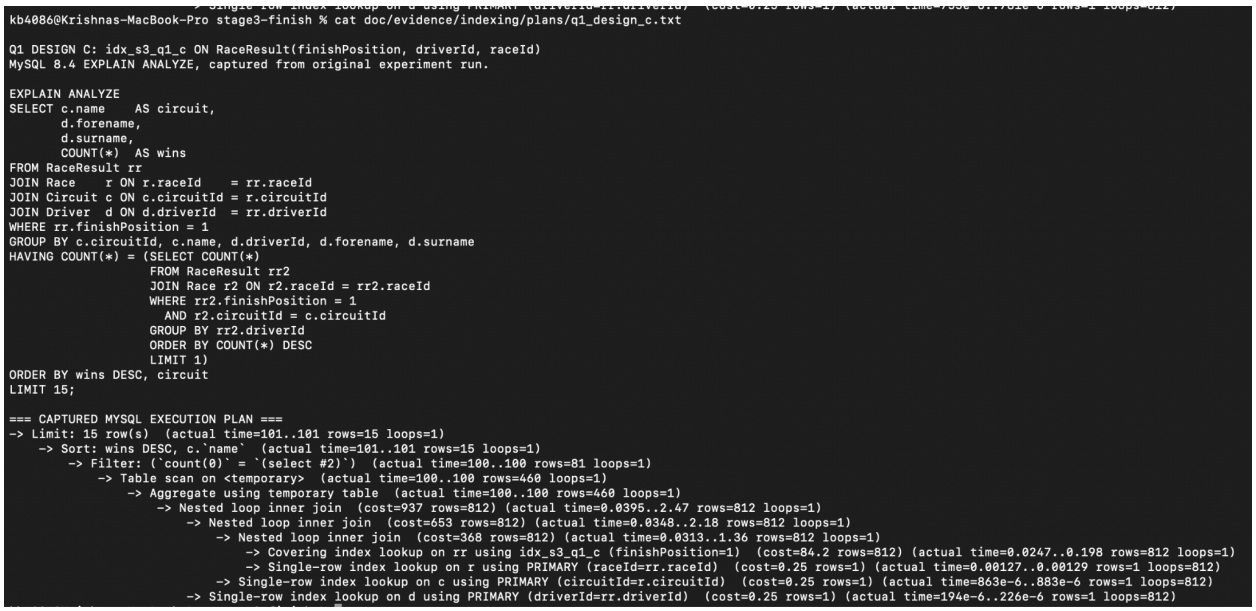
=== CAPTURED MYSQL EXECUTION PLAN ===
-> Limit: 15 row(s)  (actual time=32.4..32.4 rows=15 loops=1)
    -> Sort: wins DESC, c.`name`  (actual time=32.4..32.4 rows=15 loops=1)
        -> Filter: (`count(0)` = `(select #2)`)  (actual time=32.3..32.4 rows=81 loops=1)
            -> Table scan on <temporary>  (actual time=32.3..32.4 rows=460 loops=1)
                -> Aggregate using temporary table  (actual time=32.3..32.3 rows=460 loops=1)
                    -> Nested loop inner join  (cost=936 rows=812) (actual time=0.049..2.94 rows=812 loops=1)
                        -> Nested loop inner join  (cost=652 rows=812) (actual time=0.0442..2.17 rows=812 loops=1)
                            -> Nested loop inner join  (cost=368 rows=812) (actual time=0.0396..1.36 rows=812 loops=1)
                                -> Covering index lookup on rr using idx_s3_q1_a (finishPosition=1)  (cost=83.4 rows=812) (actual time=0.0309..0.19 rows=812 loops=1)
                                -> Single-row index lookup on r using PRIMARY (raceId=rr.raceId)  (cost=0.25 rows=1) (actual time=0.00129..0.00131 rows=1 loops=812)
                            -> Single-row index lookup on c using PRIMARY (circuitId=r.circuitId)  (cost=0.25 rows=1) (actual time=849e-6..869e-6 rows=1 loops=812)
                        -> Single-row index lookup on d using PRIMARY (driverId=rr.driverId)  (cost=0.25 rows=1) (actual time=790e-6..813e-6 rows=1 loops=812)
```



```
kb4086@Krishnas-MacBook-Pro stage3-finish % cat doc/evidence/indexing/plans/q1_design_b.txt
Q1 DESIGN B: idx_s3_q1_b ON RaceResult(finishPosition, raceId)
MySQL 8.4 EXPLAIN ANALYZE, captured from original experiment run.

EXPLAIN ANALYZE
SELECT c.name    AS circuit,
       d.forename,
       d.surname,
       COUNT(*)  AS wins
FROM RaceResult rr
JOIN Race    r ON r.raceId    = rr.raceId
JOIN Circuit c ON c.circuitId = r.circuitId
JOIN Driver  d ON d.driverId  = rr.driverId
WHERE rr.finishPosition = 1
GROUP BY c.circuitId, c.name, d.driverId, d.forename, d.surname
HAVING COUNT(*) = (SELECT COUNT(*)
                   FROM RaceResult rr2
                   JOIN Race r2 ON r2.raceId = rr2.raceId
                   WHERE rr2.finishPosition = 1
                     AND r2.circuitId = c.circuitId
                   GROUP BY rr2.driverId
                   ORDER BY COUNT(*) DESC
                   LIMIT 1)
ORDER BY wins DESC, circuit
LIMIT 15;

=== CAPTURED MYSQL EXECUTION PLAN ===
-> Limit: 15 row(s)  (actual time=32.2..32.3 rows=15 loops=1)
    -> Sort: wins DESC, c.`name`  (actual time=32.2..32.3 rows=15 loops=1)
        -> Filter: (`count(0)` = `(select #2)`)  (actual time=32.1..32.2 rows=81 loops=1)
            -> Table scan on <temporary>  (actual time=32.1..32.2 rows=460 loops=1)
                -> Aggregate using temporary table  (actual time=32.1..32.1 rows=460 loops=1)
                    -> Nested loop inner join  (cost=936 rows=812) (actual time=0.0458..2.81 rows=812 loops=1)
                        -> Nested loop inner join  (cost=652 rows=812) (actual time=0.0414..2.08 rows=812 loops=1)
                            -> Nested loop inner join  (cost=368 rows=812) (actual time=0.0382..1.3 rows=812 loops=1)
                                -> Covering index lookup on rr using idx_s3_q1_b (finishPosition=1)  (cost=83.8 rows=812) (actual time=0.031..0.188 rows=812 loops=1)
                                -> Single-row index lookup on r using PRIMARY (raceId=rr.raceId)  (cost=0.25 rows=1) (actual time=0.00122..0.00124 rows=1 loops=812)
                            -> Single-row index lookup on c using PRIMARY (circuitId=r.circuitId)  (cost=0.25 rows=1) (actual time=819e-6..839e-6 rows=1 loops=812)
                        -> Single-row index lookup on d using PRIMARY (driverId=rr.driverId)  (cost=0.25 rows=1) (actual time=755e-6..781e-6 rows=1 loops=812)
```



```
kb4086@Krishnas-MacBook-Pro stage3-finish % cat doc/evidence/indexing/plans/q1_design_c.txt

Q1 DESIGN C: idx_s3_q1_c ON RaceResult(finishPosition, driverId, raceId)
MySQL 8.4 EXPLAIN ANALYZE, captured from original experiment run.

EXPLAIN ANALYZE
SELECT c.name    AS circuit,
       d.forename,
       d.surname,
       COUNT(*)  AS wins
FROM RaceResult rr
JOIN Race    r ON r.raceId    = rr.raceId
JOIN Circuit c ON c.circuitId = r.circuitId
JOIN Driver  d ON d.driverId  = rr.driverId
WHERE rr.finishPosition = 1
GROUP BY c.circuitId, c.name, d.driverId, d.forename, d.surname
HAVING COUNT(*) = (SELECT COUNT(*)
                   FROM RaceResult rr2
                   JOIN Race r2 ON r2.raceId = rr2.raceId
                   WHERE rr2.finishPosition = 1
                     AND r2.circuitId = c.circuitId
                   GROUP BY rr2.driverId
                   ORDER BY COUNT(*) DESC
                   LIMIT 1)
ORDER BY wins DESC, circuit
LIMIT 15;

=== CAPTURED MYSQL EXECUTION PLAN ===
-> Limit: 15 row(s)  (actual time=101..101 rows=15 loops=1)
    -> Sort: wins DESC, c.`name`  (actual time=101..101 rows=15 loops=1)
        -> Filter: (`count(0)` = `(select #2)`)  (actual time=100..100 rows=81 loops=1)
            -> Table scan on <temporary>  (actual time=100..100 rows=460 loops=1)
                -> Aggregate using temporary table  (actual time=100..100 rows=460 loops=1)
                    -> Nested loop inner join  (cost=937 rows=812) (actual time=0.0395..2.47 rows=812 loops=1)
                        -> Nested loop inner join  (cost=653 rows=812) (actual time=0.0348..2.18 rows=812 loops=1)
                            -> Nested loop inner join  (cost=368 rows=812) (actual time=0.0313..1.36 rows=812 loops=1)
                                -> Covering index lookup on rr using idx_s3_q1_c (finishPosition=1)  (cost=84.2 rows=812) (actual time=0.0247..0.198 rows=812 loops=1)
                                -> Single-row index lookup on r using PRIMARY (raceId=rr.raceId)  (cost=0.25 rows=1) (actual time=0.00127..0.00129 rows=1 loops=812)
                            -> Single-row index lookup on c using PRIMARY (circuitId=r.circuitId)  (cost=0.25 rows=1) (actual time=863e-6..883e-6 rows=1 loops=812)
                        -> Single-row index lookup on d using PRIMARY (driverId=rr.driverId)  (cost=0.25 rows=1) (actual time=194e-6..226e-6 rows=1 loops=812)
```

Q3. Pit Stop Performance

Design	Index	Cost	Time	Change in cost
Baseline	Existing indexes	5,141	77.0 ms	reference
Design A	PitStop(durationMs)	7,118	99.9 ms	38.5% higher cost
Design B	PitStop(durationMs, raceId)	7,121	104.0 ms	38.5% higher cost

Design C	PitStop(raceId, durationMs)	5,141	89.1 ms	no cost change

The baseline scanned PitStop and had an estimated join cost of 5,141. Design A added an index on durationMs, but the estimated cost increased to 7,118. Design B added raceId to the index, with a slightly higher cost of 7,121. The condition durationMs < 60000 matches 10,854 out of 11,371 rows, approximately 95.5%, so the indexes provide little filtering benefit. Design C used a covering index scan but had the same estimated cost as the baseline. Since none of the three indexes reduced the estimated cost, we decided to keep the existing indexes for Q3.

Baseline: MySQL reads the whole PitStop table once, drops the few stops of 60 seconds or more, and joins the rest to RaceResult, Race and Constructor tables to group them by season and team.

Design A: MySQL now reads the pit stops through this index in order of duration instead of scanning the table. Because 95% of the entries pass the filter, it still reads nearly every pit stop, and going through an index to read almost all of a table costs more than scanning it directly, thereby increasing the cost by 38.5%.

Design B: The table’s primary key columns are already indexed, so Design A's index already contains the raceId attribute. Adding it explicitly makes no difference, and the cost matches Design A.

Design C: With raceId attribute first, the index is sorted by race, not by duration, so it anyways can't help with the durationMs filter. Hence, the cost matches the baseline.

Since none of the indexes resulted in a lower cost, we decided to stick with the default indexes for this and not add any additional index.

```
kb4086@Krishnas-MacBook-Pro stage3-finish % cat doc/evidence/indexing/plans/q3_baseline.txt

Q3 BASELINE
MySQL 8.4 EXPLAIN ANALYZE, captured from original experiment run.

EXPLAIN ANALYZE
SELECT r.year,
       con.name                        AS constructor,
       COUNT(*)                        AS stops,
       ROUND(AVG(ps.durationMs) / 1000, 3) AS avgStopSec
FROM PitStop ps
JOIN RaceResult  rr  ON rr.raceId = ps.raceId AND rr.driverId = ps.driverId
JOIN Race        r   ON r.raceId  = ps.raceId
JOIN Constructor con ON con.constructorId = rr.constructorId
WHERE ps.durationMs < 60000
GROUP BY r.year, con.constructorId, con.name
HAVING COUNT(*) >= 20
   AND AVG(ps.durationMs) < (SELECT AVG(ps2.durationMs)
                             FROM PitStop ps2
                             JOIN Race r2 ON r2.raceId = ps2.raceId
                             WHERE r2.year = r.year
                               AND ps2.durationMs < 60000)
ORDER BY r.year DESC, avgStopSec
LIMIT 15;

=== CAPTURED MYSQL EXECUTION PLAN ===
-> Limit: 15 row(s)  (actual time=77..77 rows=15 loops=1)
    -> Sort: r.`year` DESC, avgStopSec  (actual time=77..77 rows=15 loops=1)
        -> Filter: ((`count(0)` >= 20) and (??? < `(select #2)`))  (actual time=76.9..76.9 rows=84 loops=1)
            -> Table scan on <temporary>  (actual time=76.9..76.9 rows=147 loops=1)
                -> Aggregate using temporary table  (actual time=76.9..76.9 rows=147 loops=1)
                    -> Nested loop inner join  (cost=5141 rows=3790) (actual time=0.161..23.3 rows=10854 loops=1)
                        -> Nested loop inner join  (cost=3814 rows=3790) (actual time=0.156..16.3 rows=10854 loops=1)
                            -> Nested loop inner join  (cost=2488 rows=3790) (actual time=0.152..6.38 rows=10854 loops=1)
                                -> Filter: (ps.durationMs < 60000)  (cost=1161 rows=3790) (actual time=0.145..3.35 rows=10854 loops=1)
                                    -> Table scan on ps  (cost=1161 rows=11371) (actual time=0.145..2.59 rows=11371 loops=1)
                                -> Single-row index lookup on r using PRIMARY (raceId=ps.raceId)  (cost=0.25 rows=1) (actual time=139e-6..159e-6 rows=1 loops=10854)
                            -> Single-row index lookup on rr using PRIMARY (raceId=ps.raceId, driverId=ps.driverId)  (cost=0.25 rows=1) (actual time=769e-6..789e-6 rows=1 loops=10854)
                        -> Single-row index lookup on con using PRIMARY (constructorId=rr.constructorId)  (cost=0.25 rows=1) (actual time=512e-6..533e-6 rows=1 loops=10854)
```

```
kb4086@Krishnas-MacBook-Pro stage3-finish % cat doc/evidence/indexing/plans/q3_design_a.txt

Q3 DESIGN A: idx_s3_q3_a ON PitStop(durationMs)
MySQL 8.4 EXPLAIN ANALYZE, captured from original experiment run.

EXPLAIN ANALYZE
SELECT r.year,
       con.name                        AS constructor,
       COUNT(*)                        AS stops,
       ROUND(AVG(ps.durationMs) / 1000, 3) AS avgStopSec
FROM PitStop ps
JOIN RaceResult  rr  ON rr.raceId = ps.raceId AND rr.driverId = ps.driverId
JOIN Race        r   ON r.raceId  = ps.raceId
JOIN Constructor con ON con.constructorId = rr.constructorId
WHERE ps.durationMs < 60000
GROUP BY r.year, con.constructorId, con.name
HAVING COUNT(*) >= 20
   AND AVG(ps.durationMs) < (SELECT AVG(ps2.durationMs)
                             FROM PitStop ps2
                             JOIN Race r2 ON r2.raceId = ps2.raceId
                             WHERE r2.year = r.year
                               AND ps2.durationMs < 60000)
ORDER BY r.year DESC, avgStopSec
LIMIT 15;

=== CAPTURED MYSQL EXECUTION PLAN ===
-> Limit: 15 row(s)  (actual time=99.9..99.9 rows=15 loops=1)
    -> Sort: r.`year` DESC, avgStopSec  (actual time=99.9..99.9 rows=15 loops=1)
        -> Filter: ((`count(0)` >= 20) and (??? < `(select #2)`))  (actual time=99.8..99.8 rows=84 loops=1)
            -> Table scan on <temporary>  (actual time=99.8..99.8 rows=147 loops=1)
                -> Aggregate using temporary table  (actual time=99.8..99.8 rows=147 loops=1)
                    -> Nested loop inner join  (cost=7118 rows=5685) (actual time=0.0927..45.6 rows=10854 loops=1)
                        -> Nested loop inner join  (cost=5129 rows=5685) (actual time=0.0887..34.1 rows=10854 loops=1)
                            -> Nested loop inner join  (cost=3139 rows=5685) (actual time=0.0837..18.1 rows=10854 loops=1)
                                -> Filter: (ps.durationMs < 60000)  (cost=1149 rows=5685) (actual time=0.0768..3.64 rows=10854 loops=1)
                                    -> Covering index range scan on ps using idx_s3_q3_a over (NULL < durationMs < 60000)  (cost=1149 rows=5685) (actual time=0.0762..2.88 rows=10854 loops=1)
                                -> Single-row index lookup on r using PRIMARY (raceId=ps.raceId)  (cost=0.25 rows=1) (actual time=0.00119..0.00121 rows=1 loops=10854)
                            -> Single-row index lookup on rr using PRIMARY (raceId=ps.raceId, driverId=ps.driverId)  (cost=0.25 rows=1) (actual time=0.00132..0.00134 rows=1 loops=10854)
                        -> Single-row index lookup on con using PRIMARY (constructorId=rr.constructorId)  (cost=0.25 rows=1) (actual time=921e-6..941e-6 rows=1 loops=10854)
```

```
kb4086@Krishnas-MacBook-Pro stage3-finish % cat doc/evidence/indexing/plans/q3_design_b.txt

Q3 DESIGN B: idx_s3_q3_b ON PitStop(durationMs, raceId)
MySQL 8.4 EXPLAIN ANALYZE, captured from original experiment run.

EXPLAIN ANALYZE
SELECT r.year,
       con.name                       AS constructor,
       COUNT(*)                       AS stops,
       ROUND(AVG(ps.durationMs) / 1000, 3) AS avgStopSec
FROM PitStop ps
JOIN RaceResult  rr  ON rr.raceId = ps.raceId AND rr.driverId = ps.driverId
JOIN Race        r   ON r.raceId  = ps.raceId
JOIN Constructor con ON con.constructorId = rr.constructorId
WHERE ps.durationMs < 60000
GROUP BY r.year, con.constructorId, con.name
HAVING COUNT(*) >= 20
   AND AVG(ps.durationMs) < (SELECT AVG(ps2.durationMs)
                             FROM PitStop ps2
                             JOIN Race r2 ON r2.raceId = ps2.raceId
                             WHERE r2.year = r.year
                               AND ps2.durationMs < 60000)
ORDER BY r.year DESC, avgStopSec
LIMIT 15;

=== CAPTURED MYSQL EXECUTION PLAN ===
-> Limit: 15 row(s)  (actual time=104..104 rows=15 loops=1)
    -> Sort: r.`year` DESC, avgStopSec  (actual time=104..104 rows=15 loops=1)
        -> Filter: ((`count(0)` >= 20) and (??? < `(select #2)`))  (actual time=104..104 rows=84 loops=1)
            -> Table scan on <temporary>  (actual time=104..104 rows=147 loops=1)
                -> Aggregate using temporary table  (actual time=104..104 rows=147 loops=1)
                    -> Nested loop inner join  (cost=7121 rows=5685) (actual time=0.0937..49.1 rows=10854 loops=1)
                        -> Nested loop inner join  (cost=5131 rows=5685) (actual time=0.0894..36.7 rows=10854 loops=1)
                            -> Nested loop inner join  (cost=3142 rows=5685) (actual time=0.0846..19.4 rows=10854 loops=1)
                                -> Filter: (ps.durationMs < 60000)  (cost=1152 rows=5685) (actual time=0.0777..3.94 rows=10854 loops=1)
                                    -> Covering index range scan on ps using idx_s3_q3_b over (NULL < durationMs < 60000)  (cost=1152 rows=5685) (actual time=0.0769..3.09 rows=10854 loops=1)
                                -> Single-row index lookup on r using PRIMARY (raceId=ps.raceId)  (cost=0.25 rows=1) (actual time=0.00127..0.0013 rows=1 loops=10854)
                            -> Single-row index lookup on rr using PRIMARY (raceId=ps.raceId, driverId=ps.driverId)  (cost=0.25 rows=1) (actual time=0.00144..0.00146 rows=1 loops=10854)
                        -> Single-row index lookup on con using PRIMARY (constructorId=rr.constructorId)  (cost=0.25 rows=1) (actual time=994e-6..0.00102 rows=1 loops=10854)
```

```
kb4086@Krishnas-MacBook-Pro stage3-finish % cat doc/evidence/indexing/plans/q3_design_c.txt
Q3 DESIGN C: idx_s3_q3_c ON PitStop(raceId, durationMs)
MySQL 8.4 EXPLAIN ANALYZE, captured from original experiment run.

EXPLAIN ANALYZE
SELECT r.year,
       con.name                       AS constructor,
       COUNT(*)                       AS stops,
       ROUND(AVG(ps.durationMs) / 1000, 3) AS avgStopSec
FROM PitStop ps
JOIN RaceResult  rr  ON rr.raceId = ps.raceId AND rr.driverId = ps.driverId
JOIN Race        r   ON r.raceId  = ps.raceId
JOIN Constructor con ON con.constructorId = rr.constructorId
WHERE ps.durationMs < 60000
GROUP BY r.year, con.constructorId, con.name
HAVING COUNT(*) >= 20
   AND AVG(ps.durationMs) < (SELECT AVG(ps2.durationMs)
                             FROM PitStop ps2
                             JOIN Race r2 ON r2.raceId = ps2.raceId
                             WHERE r2.year = r.year
                               AND ps2.durationMs < 60000)
ORDER BY r.year DESC, avgStopSec
LIMIT 15;

=== CAPTURED MYSQL EXECUTION PLAN ===
-> Limit: 15 row(s)  (actual time=89.1..89.1 rows=15 loops=1)
    -> Sort: r.`year` DESC, avgStopSec  (actual time=89.1..89.1 rows=15 loops=1)
        -> Filter: ((`count(0)` >= 20) and (??? < `(select #2)`))  (actual time=89..89.1 rows=84 loops=1)
            -> Table scan on <temporary>  (actual time=89..89 rows=147 loops=1)
                -> Aggregate using temporary table  (actual time=89..89 rows=147 loops=1)
                    -> Nested loop inner join  (cost=5141 rows=3790) (actual time=0.148..34.3 rows=10854 loops=1)
                        -> Nested loop inner join  (cost=3814 rows=3790) (actual time=0.143..22.8 rows=10854 loops=1)
                            -> Nested loop inner join  (cost=2488 rows=3790) (actual time=0.138..16.54 rows=10854 loops=1)
                                -> Filter: (ps.durationMs < 60000)  (cost=1161 rows=3790) (actual time=0.13..3.37 rows=10854 loops=1)
                                    -> Covering index scan on ps using idx_s3_q3_c  (cost=1161 rows=11371) (actual time=0.129..2.6 rows=11371 loops=1)
                                -> Single-row index lookup on r using PRIMARY (raceId=ps.raceId)  (cost=0.25 rows=1) (actual time=147e-6..167e-6 rows=1 loops=10854)
                            -> Single-row index lookup on rr using PRIMARY (raceId=ps.raceId, driverId=ps.driverId)  (cost=0.25 rows=1) (actual time=0.00135..0.00137 rows=1 loops=10854)
                        -> Single-row index lookup on con using PRIMARY (constructorId=rr.constructorId)  (cost=0.25 rows=1) (actual time=910e-6..930e-6 rows=1 loops=10854)
```

Q4. Fastest Lap Specialists

Design	Index	Cost	Time	Change in cost
Baseline	Existing indexes	≈9,840,000	308 ms	reference
Design A	Lap(raceId, lapTimeMs)	1,041	154 ms	99.989% reduction
Design B	Lap(lapTimeMs, raceId)	859,136	263 ms	91.3% reduction
Design C	Lap(raceId, driverId, lapTimeMs)	≈10,600,000	519 ms	7.7% higher cost

To count fastest laps, the query does two jobs:

1. Find the fastest lap time in each race.

2. Go back and find which driver set that time.

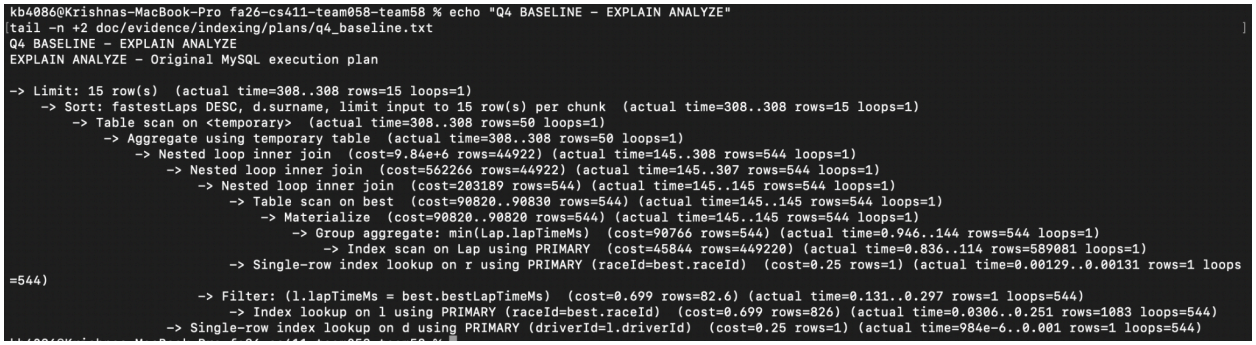
Baseline: The existing primary key organizes laps by race and driver, but it does not order lap times within each race. MySQL scans all 589,081 laps to calculate the minimum time for each race. It then retrieves the laps for each race to find those matching the minimum time.

Design A: This index works like a list sorted by race, and within each race by lap time. The fastest lap is always the first entry in each race's section. Job 1 becomes "read the first entry for each race" instead of reading every lap. Job 2 becomes a direct lookup of that race and time. Because both jobs get cheaper, the cost falls from 9.84 million to 1,041, roughly 9,000 times lower.

Design B: Same columns, opposite order. Sorted by lap time first, a race's laps are scattered across the whole index which means it still needs to read every lap. Job 2 still becomes a direct lookup. Saving only job 2's cost still cuts the total by 91.3%, but Design B's cost is about 800 times higher than Design A's.

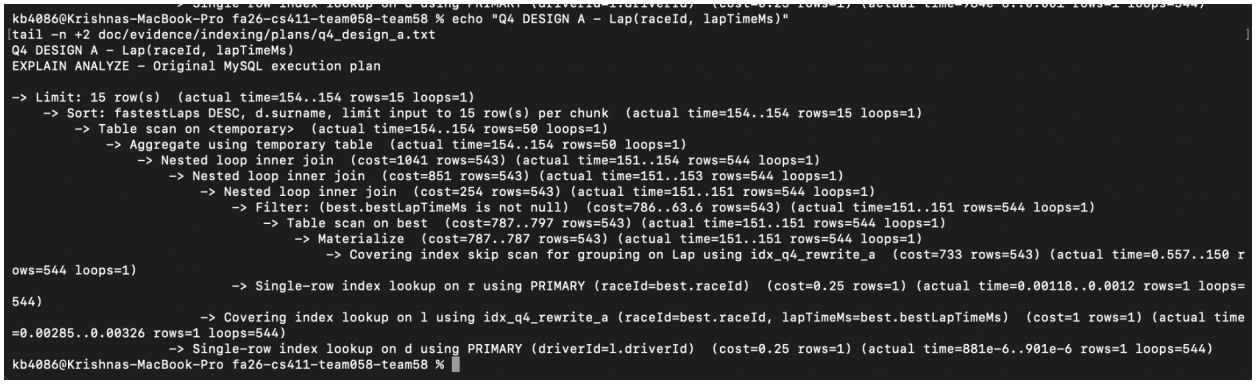
Design C: Within each race, this index is sorted by driver first, so the race's fastest lap could be anywhere in the race's section. Job 1 still reads every lap, and job 2 can't jump to a specific time because the driver isn't known yet. With neither job cheaper, the index saves nothing over the primary key, and the estimated cost rises 7.7% above the baseline.

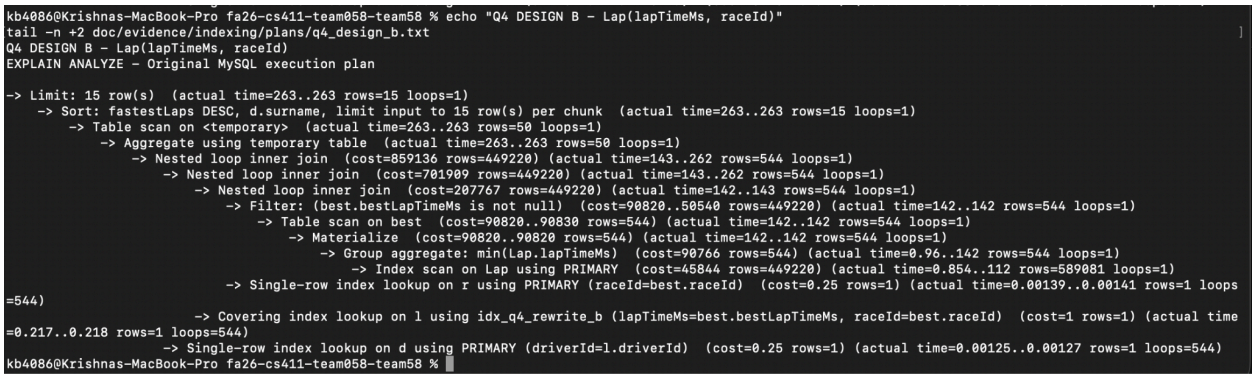
We selected Design A because the composite index on (raceId, lapTimeMs) reduced the cost of finding each race's minimum lap time and matching that time to the corresponding lap. It had the lowest estimated join cost at 1,041, compared with approximately 9,840,000 for the baseline.

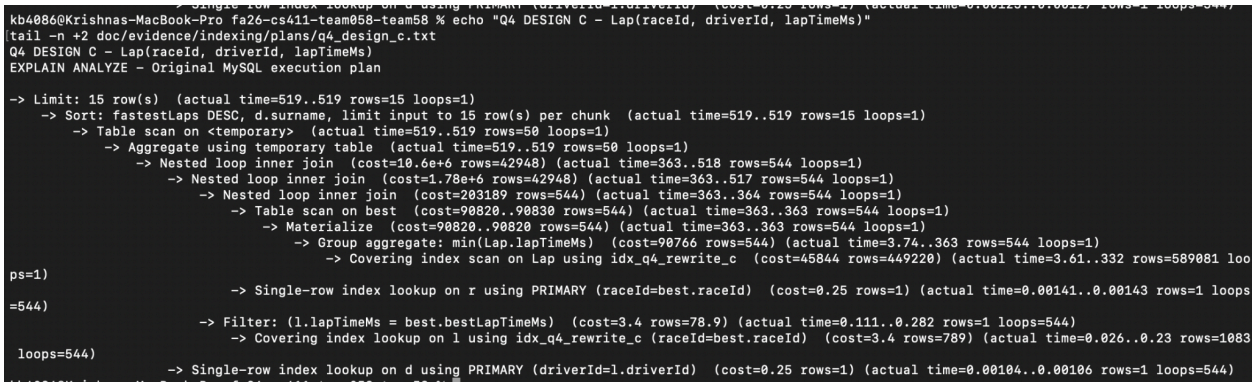


```
kb4086@Krishnas-MacBook-Pro fa26-cs411-team058-team58 % echo "Q4 BASELINE - EXPLAIN ANALYZE"
tail -n +2 doc/evidence/indexing/plans/q4_baseline.txt
Q4 BASELINE - EXPLAIN ANALYZE
EXPLAIN ANALYZE - Original MySQL execution plan

-> Limit: 15 row(s)  (actual time=308..308 rows=15 loops=1)
    -> Sort: fastestLaps DESC, d.surname, limit input to 15 row(s) per chunk  (actual time=308..308 rows=15 loops=1)
        -> Table scan on <temporary>  (actual time=308..308 rows=50 loops=1)
            -> Aggregate using temporary table  (actual time=308..308 rows=50 loops=1)
                -> Nested loop inner join  (cost=9.84e+6 rows=44922) (actual time=145..308 rows=544 loops=1)
                    -> Nested loop inner join  (cost=562266 rows=44922) (actual time=145..307 rows=544 loops=1)
                        -> Nested loop inner join  (cost=203189 rows=544) (actual time=145..145 rows=544 loops=1)
                            -> Table scan on best  (cost=90820..90830 rows=544) (actual time=145..145 rows=544 loops=1)
                                -> Materialize  (cost=90820..90820 rows=544) (actual time=145..145 rows=544 loops=1)
                                    -> Group aggregate: min(Lap.lapTimeMs)  (cost=90766 rows=544) (actual time=0.946..144 rows=544 loops=1)
                                        -> Index scan on Lap using PRIMARY  (cost=45844 rows=449220) (actual time=0.836..114 rows=589081 loops=1)
                            -> Single-row index lookup on r using PRIMARY (raceId=best.raceId)  (cost=0.25 rows=1) (actual time=0.00129..0.00131 rows=1 loops
=544)
                        -> Filter: (l.lapTimeMs = best.bestLapTimeMs)  (cost=0.699 rows=82.6) (actual time=0.131..0.297 rows=1 loops=544)
                            -> Index lookup on l using PRIMARY (raceId=best.raceId)  (cost=0.699 rows=826) (actual time=0.0306..0.251 rows=1083 loops=544)
                    -> Single-row index lookup on d using PRIMARY (driverId=l.driverId)  (cost=0.25 rows=1) (actual time=984e-6..0.001 rows=1 loops=544)
```







4. Final index selection

```
CREATE INDEX idx_rr_finish_position ON RaceResult(finishPosition);
CREATE INDEX idx_lap_race_time ON Lap(raceId, lapTimeMs);
```

We selected two indexes for our final database design. For Q1, we added idx_rr_finish_position on RaceResult(finishPosition), which reduced the estimated join cost by 73.3%. For Q4, we added idx_lap_race_time on Lap(raceId, lapTimeMs), which reduced the estimated join cost by approximately 99.989% compared with the baseline. We retained the existing indexes for Q3 because none of the three tested designs improved its estimated cost. Both selected indexes are now installed on our Google Cloud SQL database and were verified using SHOW INDEX. For Q4, the verification confirmed raceId as the first indexed column and lapTimeMs as the second, with the BTREE index marked visible.

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
RaceResult	1	idx_rr_finish_position	1	finishPosition	A	25	NULL	NULL	YES	BTREE			YES	NULL

INDEX_NAME	SEQ_IN_INDEX	COLUMN_NAME	INDEX_TYPE
idx_lap_race_time	1	raceId	BTREE
idx_lap_race_time	2	lapTimeMs	BTREE

Appendix A: Database DDL Commands

```
-- ---------- Formula 1 data relations ----------

CREATE TABLE Season (
    year    INT PRIMARY KEY,
    wikiUrl VARCHAR(2048)
);

CREATE TABLE Location (
    location VARCHAR(100) PRIMARY KEY,
    country  VARCHAR(100)
);

CREATE TABLE Circuit (
    circuitId  INT PRIMARY KEY,
    circuitRef VARCHAR(100) NOT NULL UNIQUE,
    name       VARCHAR(255),
    location   VARCHAR(100),
    latitude   DECIMAL(10,7),
    longitude  DECIMAL(10,7),
    altitudeM  INT,
    wikiUrl    VARCHAR(2048),
    FOREIGN KEY (location) REFERENCES Location(location)
);

CREATE TABLE Driver (
    driverId        INT PRIMARY KEY,
```

```
    driverRef       VARCHAR(100) NOT NULL UNIQUE,
    code            VARCHAR(3),
    permanentNumber INT,
    forename        VARCHAR(100),
    surname         VARCHAR(100),
    dateOfBirth     DATE,
    nationality     VARCHAR(100),
    wikiUrl         VARCHAR(2048)
);

CREATE TABLE Constructor (
    constructorId  INT PRIMARY KEY,
    constructorRef VARCHAR(100) NOT NULL UNIQUE,
    name           VARCHAR(255),
    nationality    VARCHAR(100),
    wikiUrl        VARCHAR(2048)
);

CREATE TABLE Race (
    raceId    INT PRIMARY KEY,
    year      INT NOT NULL,
    circuitId INT NOT NULL,
    round     INT NOT NULL,
    name      VARCHAR(255),
    raceDate  DATE,
    raceTime  TIME,
    wikiUrl   VARCHAR(2048),
    UNIQUE (year, round),
    FOREIGN KEY (year)      REFERENCES Season(year),
    FOREIGN KEY (circuitId) REFERENCES Circuit(circuitId)
);

CREATE TABLE Qualifying (
    raceId   INT,
    driverId INT,
    position INT,
    q1TimeMs INT,
    q2TimeMs INT,
    q3TimeMs INT,
    PRIMARY KEY (raceId, driverId),
    FOREIGN KEY (raceId)   REFERENCES Race(raceId),
    FOREIGN KEY (driverId) REFERENCES Driver(driverId)
);

CREATE TABLE DriverStanding (
```

```
    year     INT,
    driverId INT,
    position INT,
    points   DECIMAL(10,2),
    PRIMARY KEY (year, driverId),
    FOREIGN KEY (year)     REFERENCES Season(year),
    FOREIGN KEY (driverId) REFERENCES Driver(driverId)
);

CREATE TABLE ConstructorStanding (
    year          INT,
    constructorId INT,
    position      INT,
    points        DECIMAL(10,2),
    PRIMARY KEY (year, constructorId),
    FOREIGN KEY (year)          REFERENCES Season(year),
    FOREIGN KEY (constructorId) REFERENCES Constructor(constructorId)
);

CREATE TABLE RaceResult (
    raceId         INT,
    driverId       INT,
    constructorId  INT NOT NULL,
    carNumber      INT,
    gridPosition   INT,
    finishPosition INT,
    positionOrder  INT,
    points         DECIMAL(10,2),
    lapsCompleted  INT,
    raceTimeMs     INT,
    status         VARCHAR(100),
    PRIMARY KEY (raceId, driverId),
    FOREIGN KEY (raceId)        REFERENCES Race(raceId),
    FOREIGN KEY (driverId)      REFERENCES Driver(driverId),
    FOREIGN KEY (constructorId) REFERENCES Constructor(constructorId)
);

CREATE TABLE Lap (
    raceId       INT,
    driverId     INT,
    lapNumber    INT,
    position     INT,
    lapTimeMs    INT,
    sector1Ms    INT,
    sector2Ms    INT,
```

```
    sector3Ms    INT,
    tyreCompound VARCHAR(30),
    tyreLifeLaps INT,
    trackStatus  VARCHAR(20),
    speedTrapKph DECIMAL(7,3),
    PRIMARY KEY (raceId, driverId, lapNumber),
    FOREIGN KEY (raceId, driverId) REFERENCES RaceResult(raceId,
driverId)
);

CREATE TABLE PitStop (
    raceId     INT,
    driverId   INT,
    lapNumber  INT,
    durationMs INT,
    timeOfDay  TIME,
    PRIMARY KEY (raceId, driverId, lapNumber),
    FOREIGN KEY (raceId, driverId, lapNumber) REFERENCES Lap(raceId,
driverId, lapNumber)
);

CREATE TABLE TelemetrySample (
    raceId      INT,
    driverId    INT,
    lapNumber   INT,
    offsetMs    INT,
    speedKph    DECIMAL(7,3),
    rpm         INT,
    gear        INT,
    throttlePct DECIMAL(5,2),
    brakeOn     BOOLEAN,
    drsState    INT,
    posX        DECIMAL(12,3),
    posY        DECIMAL(12,3),
    PRIMARY KEY (raceId, driverId, lapNumber, offsetMs),
    FOREIGN KEY (raceId, driverId, lapNumber) REFERENCES Lap(raceId,
driverId, lapNumber)
);

-- ---------- Application relations ----------

CREATE TABLE User (
    userId       INT PRIMARY KEY,
    username     VARCHAR(50)  NOT NULL UNIQUE,
    email        VARCHAR(254) NOT NULL UNIQUE,
```

```
    passwordHash VARCHAR(255) NOT NULL,
    country      VARCHAR(100),
    isAdmin      BOOLEAN NOT NULL DEFAULT FALSE,
    createdAt    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Question (
    questionId  INT PRIMARY KEY,
    authorId    INT,
    raceId      INT,
    driverId    INT,
    lapNumber   INT,
    text        TEXT NOT NULL,
    format      VARCHAR(30),
    category    VARCHAR(100),
    difficulty  VARCHAR(20),
    explanation TEXT,
    visibility  VARCHAR(20),
    sourceKey   VARCHAR(255) UNIQUE,
    createdAt   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (authorId) REFERENCES User(userId),
    FOREIGN KEY (raceId, driverId, lapNumber) REFERENCES Lap(raceId,
driverId, lapNumber)
);

CREATE TABLE Quiz (
    quizId        INT PRIMARY KEY,
    ownerId       INT,
    driverId      INT,
    constructorId INT,
    title         VARCHAR(255) NOT NULL,
    description   TEXT,
    quizType      VARCHAR(30),
    visibility    VARCHAR(20),
    category      VARCHAR(100),
    eraStartYear  INT,
    eraEndYear    INT,
    createdAt     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (ownerId)       REFERENCES User(userId),
    FOREIGN KEY (driverId)      REFERENCES Driver(driverId),
    FOREIGN KEY (constructorId) REFERENCES Constructor(constructorId)
);

CREATE TABLE QuizQuestion (
    quizId     INT,
```

```
    questionId INT,
    position   INT,
    PRIMARY KEY (quizId, questionId),
    FOREIGN KEY (quizId)     REFERENCES Quiz(quizId),
    FOREIGN KEY (questionId) REFERENCES Question(questionId)
);

CREATE TABLE AnswerOption (
    questionId INT,
    label      VARCHAR(10),
    text       TEXT NOT NULL,
    isCorrect  BOOLEAN NOT NULL,
    PRIMARY KEY (questionId, label),
    FOREIGN KEY (questionId) REFERENCES Question(questionId)
);

CREATE TABLE QuizAttempt (
    attemptId   INT PRIMARY KEY,
    quizId      INT NOT NULL,
    userId      INT NOT NULL,
    startedAt   TIMESTAMP NULL,
    completedAt TIMESTAMP NULL,
    FOREIGN KEY (quizId) REFERENCES Quiz(quizId),
    FOREIGN KEY (userId) REFERENCES User(userId)
);

CREATE TABLE AttemptAnswer (
    attemptId     INT,
    questionId    INT,
    selectedLabel VARCHAR(10),
    timeTakenMs   INT,
    PRIMARY KEY (attemptId, questionId),
    FOREIGN KEY (attemptId) REFERENCES QuizAttempt(attemptId),
    FOREIGN KEY (questionId) REFERENCES Question(questionId),
    FOREIGN KEY (questionId, selectedLabel) REFERENCES
AnswerOption(questionId, label)
);

CREATE TABLE Favorite (
    userId     INT,
    questionId INT,
    savedAt    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (userId, questionId),
    FOREIGN KEY (userId)     REFERENCES User(userId),
    FOREIGN KEY (questionId) REFERENCES Question(questionId)
```

```
);
```